

为非确定性 AI 智能体构建确定性控制系统

一门完整大师课：用 Claude、记忆架构、AST 防护栏、TDA 自行修复循环和复合多智能体团队，工程化可靠、可观测、安全的生产级智能体驾驭系统。



课程讲师

Ken Huang

客座教授，[旧金山大学（USF）](#)

首席执行官，[DistributedApps.ai](#)

Substack: kenhuangus.substack.com

LinkedIn: linkedin.com/in/kenhuang8

确定性驾驭

记忆、范围沙箱与 AST 钩子

测试驱动的可靠性

Pytest 反馈与防回归测试

多智能体系统

规划者、实现者与复核者 worktree

五道门记分卡

生产就绪审计



KEN HUANG, CISSP — 讲师与 AI 安全研究者

《Harness Engineering》与《MAESTRO》作者 | 旧金山大学 (USF) 客座教授
CSA 院士与联合主席 | OWASP AIVSS 负责人 | AIUC-1 联盟 | Schmidt Sciences

- ◆ AI 图书作者与演讲者 (《Harness Engineering》《MAESTRO》)
- ◆ 旧金山大学 (USF) 客座教授: <https://www.usfca.edu/faculty/ken-huang>
- ◆ OWASP AIVSS 项目负责人
- ◆ CSA 院士, 并担任两个 CSA AI 安全工作组联合主席
- ◆ OWASP Top 10 for LLM Applications 核心成员
- ◆ AIUC-1 联盟成员
- ◆ Schmidt Sciences 基金评审成员
- ◆ EC-Council 讲师 — 面向网络安全的生成式 AI
- ◆ DistributedApps.ai 首席执行官: <https://distributedapps.ai/>
- ◆ Substack: <https://kenhuangus.substack.com/> | LinkedIn: <https://www.linkedin.com/in/kenhuang8/>

AI 图书与学术出版物 (Springer · Cambridge · Wiley · Packt)

[亚马逊作者页 →](#)



Agentic AI
SPRINGER



Beyond AI
SPRINGER



GenAI Security
SPRINGER



Securing Agents
SPRINGER



Web3 & Economy
CAMBRIDGE



Blockchain Web3
WILEY



Harness Eng.
精研



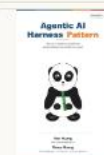
OpenClaw AI
PACKT



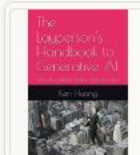
Agentic Claude
CLAUDE AI



LLM Patterns
PACKT



AI Harness
模式



GenAI Guide
手册

🏠 传统软件工程

确定性 IT 系统：显式逻辑与可重复输出

SDLC · Agile · 架构
DevOps · CI/CD · 测试分层

SRE 与安全 SDLC
AppSec · SAST/DAST

公式：f(x) → y [严格的输入输出可重复性]

范式
转变

🛡️ 面向智能体 AI 的驾驭工程

非确定性系统：模型 + 涌现行为

概率核心
模型 · 提示 · 上下文 · 工具

确定性控制驾驭层
记忆 · 沙箱 · 钩子 · TDA

目标：可靠 · 可观测 · 安全 · 可治理 · 可运维

🏠 传统软件工程

SDLC、DevOps、CI/CD、测试分层与 SRE

确定性系统

🔧 系统领域：面向确定性 IT 系统构建。在这类系统中，显式逻辑与算法运行于可预测的计算机硬件。

⚙️ 框架与实践：SDLC、Agile、架构模式、CI/CD 流水线、单元 / 集成测试分层、SRE，以及安全 SDLC / AppSec。

🔑 核心原则：确定性可重复 — 相同输入和状态严格产生相同输出 (f(x) → y)。

🛡️ 面向智能体 AI 的驾驭工程

面向概率智能体的确定性控制系统

非确定性 + 驾驭层

💡 概率涌现：行为由基础模型，加上提示、动态上下文、记忆、工具、循环、策略和多智能体团队，以概率方式产生。

🔧 确定性控制环境：为周围建立脚手架：记忆 (CLAUDE.md / AGENTS.md)、路径沙箱、前 / 后 AST 钩子、Pytest TDA 循环和权限网关。

🎯 五项生产目标：把概率模型输出转变成这样的系统：可靠, 可观测, 安全, 可治理，以及 可运维。

第一部分：基础与控制

模块 1-5：核心脚手架、规范驱动开发与网关

确定性执行与拦截

第二部分：可靠性与团队

模块 6-10：TDA、MCP、多智能体与审计

五道门生产就绪记分卡

第一部分：基础与控制

- 模块 1：为什么需要驾驭工程
- 模块 2：核心驾驭栈
- 模块 3：规范驱动开发
- 模块 4：防护栏与钩子
- 模块 5：权限网关

第二部分：可靠性与团队

- 模块 6：测试作为可靠性
- 模块 7：智能体扩展 (Skills、Plugins、MCP)
- 模块 8：复合工程
- 模块 9：五步 SOP
- 模块 10：原则与审计

概率模型与确定性控制系统

- ◆ 模型能力不等于生产可靠性：生成很快，没有驾驭控制时后续执行跟不上。
- ◆ 概率模型层：提出代码；会出现遗忘、幻觉、偏离和重试陷阱。
- ◆ 确定性驾驭层：执行沙箱、**AST** 解析、循环检测和密钥扫描。
- ◆ 主要故障模式：上下文衰减、重复执行循环、破坏性 **shell** 命令、未经核验的编辑。
- ◆ 核心要求：驾驭层是控制系统。优先保证可信，再考虑生成速度。

环境资源与执行指导

环境配置 (.env.example):
[.env.example \(API 密钥与模型提供商\)](#)

交互式实验演示:
[模块 01: 执行前拦截与循环检测](#)

为什么使用虚拟环境:
Each module executes `pytest` via `sys.executable` — 确保测试运行在专用项目环境中, 不会因为系统 PATH 上的另一个 Python 而假通过。

完整准备与多模型密钥:
[Packt 大师课 README \(准备、API 密钥、浏览器附加项\)](#)

统一环境准备（只需做一次）

用这一套准备, 10 个模块实验都以确定性方式执行

步骤 1: 克隆仓库

必需 (.git)

克隆 git 仓库 (不要用 ZIP: 模块 8 使用 git worktree, 需要 .git):
`$ git clone https://github.com/kenhuangus/packt-harness.git`

复制

步骤 2: 进入目录并创建虚拟环境

Python 3.10+

进入 `packt-harness`, 再创建隔离的 `.venv`:

`$ cd packt-harness`

复制

Windows (PowerShell / 命令提示符):

`PS> python -m venv .venv`

复制

macOS / Linux (Bash / Zsh):

`$ python3 -m venv .venv`

复制

步骤 3: 激活虚拟环境

每个会话都需要

激活 `.venv` so `python` and `pip` 指向项目解释器 (每个新终端都要重新执行):

Windows (PowerShell):

`PS> .venv\Scripts\Activate.ps1`

复制

Windows (命令提示符):

`> .venv\Scripts\activate.bat`

复制

macOS / Linux (Bash / Zsh):

`$ source .venv/bin/activate`

复制

步骤 4: 以可编辑模式安装

-e .

安装驾驶工程包 (激活之后, 或直接使用 `.venv` 解释器路径):

激活之后 (全部平台):

`$ pip install -e .`

复制

Windows (未激活):

`PS> .venv\Scripts\python.exe -m pip install -e .`

复制

macOS / Linux (未激活):

`$ .venv/bin/python -m pip install -e .`

复制

步骤 5: 检查并核验全部模块

14 项通过 ✓

在全部模块上运行驾驶测试运行器 (预期 Summary: 14 passed, 0 failed):

`$ python run_all_modules.py`

复制

模块 1 代码实验：执行前拦截与循环检测

第 7 张 / 共 47 张

```
course_implementation/module_01_why_harness_engineering/harness_v... 复制 PYTHON

1 class HarnessedAgent:
2     def __init__(self, max_retries: int = 2) → None:
3         self.max_retries = max_retries
4         self.command_history: list[str] = []
5
6     def pre_execution_hook(self, command: str) → None:
7         # Line 7-10: Deterministic regex guard against destructive shell calls
8         要点 for pattern in FORBIDDEN_PATTERNS: # ['rm -rf', 'sudo', 'chmod 777']
9         要点 if re.search(pattern, command):
10        要点 raise PermissionError(f"BLOCKED BY PRE-HOOK: '{pattern}' detected")
11
12    def loop_detector(self, command: str) → None:
13        # Line 13-18: Sliding-window memory breaks infinite retry loops
14        self.command_history.append(command)
15        要点 if len(self.command_history) ≥ self.max_retries:
16        要点 recent = self.command_history[-self.max_retries:]
17        要点 if len(set(recent)) == 1:
18        要点 raise RuntimeError(f"LOOP DETECTOR: '{command}' repeated {self.max_retries} times")
```

文件：

course_implementation/module_01_why_harness_engineering/harness_vs_model...

[第 8-10 行] 执行前拦截：用正则匹配破坏性命令（`rm -rf`、`sudo`），在创建子进程之前抛出 `PermissionError`。

[第 15-18 行] 循环检测器：滑动窗口记录最近命令，达到 `max_retries` 后抛出 `RuntimeError`，停止无限重试。

已核验的执行：没有驾驭层的智能体会删除 `var/log/app.log` 并无限循环；有驾驭层的智能体安全停止，且 `log_survived=True`。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_01_why_harness_engineering/harness_vs_model_demo.py



模块 2：驾驭架构的 5 个核心支柱

面向编码智能体的架构脚手架

- ◆ 支柱 1：记忆（用户编写与模型生成，短期与长期）
- ◆ 支柱 2：受限工具（最小权限允许列表，以及 `Path.is_relative_to()` 沙箱）
- ◆ 支柱 3：确定性钩子（`PascalCase` 的 `PreToolUse` 与 `PostToolUse` `AST` / 密钥防护）
- ◆ 支柱 4：`Token` 预算（`20/20/50/10` 分配，以及头尾日志压缩）
- ◆ 支柱 5：结构化追踪（`events.jsonl` 只追加、不可变的审计记录）

1. CLAUDE.md (用户)

持久指导

规则、架构与标准

会话开始时完整载入

2. Auto Memory (智能体)

自动积累的发现

MEMORY.md 与主题笔记

前 200 行 / 25KB

3. 五层作用域

解析顺序

托管 → 用户 → 项目

本地 .md → .claude/rules/

4. 导入与互操作

模块化 @path 语法

@AGENTS.md 桥接

双记忆系统：CLAUDE.md 与 Auto Memory

Claude Code v2.1+

每次 Claude Code 会话从干净的上下文窗口开始。两套互补机制把知识带到后续会话：

特性	CLAUDE.md 文件	Auto Memory (MEMORY.md)
谁来编写	你（开发者、团队、组织）	Claude（自动）
内容	编码标准、工作流、架构	构建命令、调试笔记、偏好
作用域	托管策略、用户、项目、本地	Per repo (~/.claude/projects/), shared worktrees
载入位置	每个会话开始时完整载入	前 200 行或 25KB 的 MEMORY.md
压缩	根文件在此后重新注入 /compact	主题笔记按工具按需读取

强制执行与上下文： 记忆文件提供行为指导。要阻断动作，不依赖模型自行决定，请用确定性的 PreToolUse 钩子。

现场学员演示 (MEMORY.md)：

C:\Users\kenhu\claude\projects\C--Users-kenhu-security-tools\memory\MEMORY.md

作用域层次、规则与互操作

解析顺序

四层加载优先级（从最宽到最具体）：

1. 托管策略： /etc/claude-code/CLAUDE.md or C:\Program Files\ClaudeCode\CLAUDE.md
2. 用户指令： ~/.claude/CLAUDE.md （跨全部仓库的个人偏好）
3. 项目指令： ./CLAUDE.md or ./claude/CLAUDE.md （git 中的团队标准）
4. 本地指令： ./CLAUDE.local.md （个人沙箱偏好；已被 git 忽略）

模块化规则与跨智能体互操作：

- 路径范围规则： .claude/rules/*.md with paths: ["src/api/**/*.ts"] 只在打开匹配文件时加载。
- AGENTS.md 互操作： Claude Code 读取 CLAUDE.md ; use @AGENTS.md or ln -s AGENTS.md CLAUDE.md 以共享指令。

官方文档与现场参考：

- Docs Index: <https://code.claude.com/docs/llms.txt>
- Memory Guide: <https://code.claude.com/docs/en/memory>
- Live Example: [MEMORY.md \(真实演示\)](#)
- Advanced Research & Benchmarks: [智能体记忆驱动演示](#)

最小权限

- ◆ 只允许需要的工具
- ◆ 写入限制在工作区内
- ◆ 阻断路径穿越



6 类拦截上的确定性防护栏

- ◆ **31 个生命周期钩子**：Claude Code 在 `.claude/settings.json` 中提供 **31 个确定性 PascalCase** 生命周期事件，对每一轮、每一条 **shell** 命令、每一个 **worktree** 和每一次记忆压缩给出可编程控制。
- ◆ **会话与轮次 (7 个钩子)**：SessionStart、Setup (CI/脚本标志)、UserPromptSubmit、UserPromptExpansion、Stop、StopFailure、SessionEnd (管理环境初始化和轮次收尾)。
- ◆ **工具与执行循环 (6 个钩子)**：PreToolUse (安全阻断；退出码 **2** 阻止执行)、PermissionRequest、PermissionDenied、PostToolUse (AST 解析与密钥过滤)、PostToolUse Failure、PostToolBatch。
- ◆ **子智能体与团队编排 (5 个钩子)**：SubagentStart、SubagentStop (质量门)、TeammateIdle、TaskCreated、TaskCompleted (接受工作前执行“完成定义”检查)。
- ◆ **环境、配置与 worktree (7 个钩子)**：InstructionsLoaded (CLAUDE.md / AGENTS.md)、ConfigChange、CwdChanged、DirectoryAdded、FileChanged、WorktreeCreate、WorktreeRemove (临时 git worktree 隔离)。
- ◆ **上下文压缩 (2 个钩子)**：PreCompact (拦截 token 压力)、PostCompact (记忆压缩后重新注入关键项目规则)。
- ◆ **MCP 与系统交互 (4 个钩子)**：Notification (Slack / 桌面提醒)、MessageDisplay、Elicitation、ElicitationResult (把用户回复送往 MCP 服务器之前先做清理)。

⚡ 支柱 4：上下文预算与压缩

20/20/50/10 分配（记忆·规范·工作区·余量）

头尾压缩：保留前 5 轮与后 15 轮

消除上下文偏离，并防止内存耗尽

📁 支柱 5：Claude Code JSONL 记录

~/claude/projects/<encoded-path>/<session-id>.jsonl

只追加事件：user · assistant · tool_use · tool_result · usage

直接用 jq / Python 检查 · Token 归属与审计

上下文分配与不可变审计日志（~/claude/projects/ /.jsonl）

◆ 支柱 4：上下文 Token 预算与压缩（core_harness_stack.py 中的 ContextTokenBudgeter）：

- › 20/20/50/10 Token 分配：20% 记忆（CLAUDE.md），20% 规范（SPEC.md），50% 工作区（当前代码 / diff），10% 输出余量。
- › 头尾压缩：保留对话的前 5 行 / 轮和后 15 行 / 轮，略去中间噪声日志，避免上下文偏离。

◆ 支柱 5：结构化追踪与 Claude Code JSONL 记录架构：

- › 文件布局与存储：位于 ~/claude/projects/.jsonl（每个会话 UUID 一个文件，只追加，每行一个 JSON 对象）。
- › 常见顶层字段：type（user / assistant / system）、uuid、parentUuid、timestamp（ISO 8601）、sessionId、cwd、gitBranch、version。
- › 事件类型与载荷：type: 'user'（提示与 tool_result 块），type: 'assistant'（text、thinking、tool_use 块，以及 message.usage 的 token 记账）。
- › 直接审计检查（jq / Python）：用 jq -c 'select(.type=="assistant")|.message.content[]|select(.type=="tool_use")|{name,input}' 扫描工具调用，或用 .message.usage 查询 token 用量。

◆ [🔗 实验演示：在 GitHub 打开本模块](#)

```
course_implementation/module_02_core_harness_stack/core_harness_stack.py 复制 PYTHON

1 class CoreHarnessStack:
2     def validate_tool_permission(self, tool_name: str, target_path: str = None) → bool:
3         # Line 3-6: Least-privilege tool allowlist
4         要点 allowed_tools = ["read_file", "write_file", "run_test", "list_dir"]
5         要点 if tool_name not in allowed_tools:
6             self.log_event("PERMISSION_DENIED", {"tool": tool_name})
7             return False
8
9         # Line 9-14: Path sandboxing blocks directory traversal (../)
10        if target_path:
11        要点 workspace = Path(self.workspace_root).resolve()
12        要点 abs_target = Path(target_path).resolve()
13        要点 if not abs_target.is_relative_to(workspace):
14            self.log_event("PATH_TRAVERSAL_BLOCKED", {"path": str(abs_target)})
15            return False
16        return True
17
18    def run_post_edit_hook(self, file_path: str, code_content: str) → bool:
19    要点 # Line 19-21: Secret leak scanner blocks hardcoded credentials
20    要点 if "sk-proj-" in code_content:
21        self.log_event("SECURITY_VIOLATION", {"file": file_path, "issue": "Secret leak"})
22        return False
23    return True
```

文件：

course_implementation/module_02_core_harness_stack/core_harness_stack.py

[第 4-5 行] 工具权限允许列表：用显式允许的工具登记表替换不受约束的智能体能力。

[第 11-13 行] 路径沙箱：用 Path.resolve().is_relative_to() 阻止目录穿越 (../) 和外部访问。

[第 19-20 行] 编辑后密钥扫描：在写入磁盘前检查拟议代码中的硬编码 API 密钥 (sk-proj-)。

审计可观测性：所有工具请求和钩子决定都写入 events.jsonl，并带 ISO 时间戳。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_02_core_harness_stack/core_harness_stack.py ↗

1. SPEC.md 契约

不可变文件范围

验收标准

2. 允许的范围

`allowed_files` 检查

阻断 `database.py`

3. 非目标扫描

范围违规

拒绝功能蔓延

4. AST 预校验

`ast.parse()` 语法检查

零语法错误

提示为什么会失败，机器可读规范为什么能成立

- ◆ 提示含糊、规格不足，长对话中容易发生上下文偏离。
- ◆ 机器可核验的契约：`SPEC.md` 定义允许的文件范围、模式和非目标。
- ◆ 硬性范围边界：**SpecVerifier** 拒绝超范围 **diff**，并阻断未批准的写入。
- ◆ 非目标过滤：在代码保存前阻断架构蔓延和未经请求的依赖。
- ◆ 自动核验：用 `ast.parse()` 检查语法，并在交付前运行 `pytest`。

必需章节

- ◆ 目标与范围（允许 / 禁止的文件）
- ◆ 明确的非目标（不做数据库连接池，不做 **OAuth2**）
- ◆ 输入 / 输出模式（**token** -> **dict**）
- ◆ 验收标准（**AC-o1** 到 **AC-o4**）

硬边界

- ◆ 允许的文件: **auth_validator.py**、**test_auth.py**
- ◆ 禁止的文件: **database.py**、**config/settings.py**
- ◆ 非目标: 拒绝 **connect_db** 和 **OAuth2** 轮换
- ◆ 沙箱: 用 **Path.is_relative_to()** 限定

机器可核验的契约

- ◆ **AC-01:** 导出 `validate_jwt(token: str) -> dict`
- ◆ **AC-02:** 对有效 **JWT** 返回 `valid=True, user_id=123`
- ◆ **AC-03:** 对过期 **JWT** 返回 `valid=False, error=EXPIRED`
- ◆ **AC-04:** 通过 `test_auth.py` 中的全部自动测试

```
course_implementation/module_03_spec_driven_development/spec_driv... 复制 PYTHON

1 class SpecVerifier:
2     def attempt_write(self, target_file: str, code: str) → tuple[bool, str]:
3         relative = self._normalized(target_file)
4         dest = (self.workspace / relative).resolve()
5
6         # Line 6-8: Strict scope boundary - only files listed in SPEC.md allow
7         # Line 6-8: Strict scope boundary - only files listed in SPEC.md allow
8         if relative not in self.allowed_files or relative in self.forbidden_files:
9             return False, "SCOPE_VIOLATION" # Blocks database.py
10
11         # Line 10-13: Non-goals filter blocks unrequested architectural sprawl
12         # Line 10-13: Non-goals filter blocks unrequested architectural sprawl
13         lowered = code.lower()
14         if "database" in lowered or "connect_db" in lowered or "oauth2" in lowered:
15             return False, "NON_GOAL_VIOLATION" # Blocks database connections
16
17         # Line 16-19: Syntax validation guarantees code compiles before save
18         dest.parent.mkdir(parents=True, exist_ok=True)
19         dest.write_text(code, encoding="utf-8")
20
21         ast.parse(code, filename=str(dest))
22         return True, str(dest)
```

文件：

course_implementation/module_03_spec_driven_development/spec_driven_veri

[第 6-7 行] 范围白名单：文件写入必须匹配 `SPEC.md` 的 `allowed_files`；拒绝未批准文件（例如 `database.py`）。

[第 10-12 行] 非目标过滤：对照明确的非目标扫描 diff，防止范围蔓延（例如在 JWT 校验器里加入数据库逻辑）。

[第 18 行] AST 解析门：保存前运行 `ast.parse()`，使语法损坏的代码不能进入工作区。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_03_spec_driven_development/spec_driven_verifier.py



1. 系统提示

常驻指导

CLAUDE.md / AGENTS.md

2. 工具模式

JSON Schema 类型

严格的参数检查

3. 前 / 后钩子

PreToolUse 拒绝 shell

PostToolUse AST / 密钥

4. 操作系统沙箱

Path is_relative_to

进程隔离

安全工具执行的纵深防御

- ◆ 第 1 层（系统提示）：CLAUDE.md 和 AGENTS.md 中的常驻项目规则与记忆。
- ◆ 第 2 层（工具模式）：强类型 JSON Schema 参数校验。
- ◆ 第 3 层（钩子）：PascalCase 的 PreToolUse 标志拒绝，以及 PostToolUse 的 AST / 密钥扫描。
- ◆ 第 4 层（操作系统沙箱）：解析后路径隔离，以及进程级最小权限。
- ◆ 确定性执行：安全策略写在代码里，由程序执行，不交给模型自行决定。
- ◆  实验演示：在 GitHub 打开本模块

```
course_implementation/module_04_guardrails_and_hooks/guardrails_e... 复制 PYTHON

1 class ClaudeCodeHookInterceptor:
2     def intercept_pre_tool_use(self, tool_name: str, tool_input: dict) → tuple[bool, str]:
3         # Line 3-7: Intercepts dangerous CLI flags that attempt to bypass permissions
4         command = tool_input.get("command", "")
5         要点 if tool_name == "Bash":
6         要点 for flag in ["--dangerously-skip-permissions", "-y", "--force-all"]:
7             if flag in command:
8                 return False, f"CLI flag '{flag}' prohibited by policy."
9             return True, "PreToolUse check passed."
10
11     @classmethod
12     要点 def permission_output(cls, decision: str, reason: str) → dict:
13     要点 # Line 12-16: Emits standard Claude Code PreToolUse JSON schema
14     要点 return {"hookSpecificOutput": {
15     要点     "hookEventName": "PreToolUse",
16     "permissionDecision": decision, # 'allow' | 'deny' | 'ask' | 'defer'
17     "permissionDecisionReason": reason,
18 }}
19
20 class GuardrailsEngine:
21     def audit_ast_and_secrets(self, file_path: str, code_content: str) → tuple[bool, str]:
22     要点 # Line 21-25: Multi-vendor credential scan + AST syntax parser
23     要点 ast.parse(code_content, filename=file_path)
24     if re.search(r"(sk-proj-[A-Za-z0-9-]{20,}|AKIA[0-9A-Z]{16})", code_content):
25         return False, ["CRITICAL: Hardcoded API secret key detected!"]
26     return True, ["AST syntax valid. Zero secrets."]
```

文件:

course_implementation/module_04_guardrails_and_hooks/guardrails_engine.py

[第 5-6 行] 标志绕过阻断：在调用 shell 之前拦截 --dangerously-skip-permissions 这类标志。

[第 12-15 行] Claude Code PreToolUse 模式：输出标准 permissionDecision JSON (allow、deny、ask、defer)。

[第 22-23 行] AST 与正则密钥审计：解析语法，并在写文件前扫描供应商凭证 (OpenAI sk-proj-、AWS IAM AKIA)。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_04_guardrails_and_hooks/guardrails_engine.py ↗

低风险

read_file, list_dir, grep

✓ 自动批准

中风险

write_file, run_test

✓ 记录并批准

高风险

pip_install

⚠ 记录意图并告警

严重风险

git_push, db_drop

🚫 approvals.json 账本

按风险分级的批准矩阵与执行动作

风险	工具	本演示中的动作
低	read_file , list_dir, grep	自动批准
中	write_file , run_test	记录并批准
高	pip_install	额外记录，仍然允许
严重	git_push, db_drop	除非 user_auto_approve=True，否则阻断

🔗 实验演示：在 **GitHub** 打开本模块

```
course_implementation/module_05_break_and_qna/permission_escalati... 复制 PYTHON

1 class PermissionEscalationGateway:
2     def evaluate_request(self, tool_name: str, params: dict, request_id: str) -> dict:
3         risk = self.risk_matrix.get(tool_name, "HIGH")
4
5         # Line 5-7: LOW risk (reads) -> Auto-approved
6         if risk == "LOW":
7             return {"allowed": True, "content": (self.workspace / params["path"]).read_text()}
8
9         # Line 9-11: MEDIUM risk (writes) -> Logged and executed in sandbox
10        if risk == "MEDIUM":
11            (self.workspace / params["path"]).write_text(params["content"], encoding="utf-8")
12            return {"allowed": True, "path": str(self.workspace / params["path"])}
13
14        # Line 13-20: CRITICAL risk (git_push) -> Strictly gated by approvals
15        if risk == "CRITICAL":
16            approvals = self._approvals() # reads approvals.json
17            if request_id not in approvals:
18                self._log({"event": "ESCALATION_BLOCKED", "request_id": request_id})
19                return {"allowed": False, "reason": "missing_approval"}
20            pending = {"request_id": request_id, "approved_by": approvals[request_id]}
21            (self.workspace / "pending_push.json").write_text(json.dumps(pending, indent=2))
22            return {"allowed": True, "pending_push": str(self.workspace / "pending_push.json")}
```

文件：

course_implementation/module_05_break_and_qna/permission_escalation_gateway.py

[第 5 行] 风险分级：自动批准安全的只读操作（低风险），不拖慢智能体。

[第 9 行] 沙箱写入：记录并允许工作区内的文件变更（中风险）。

[第 14-18 行] 批准账本：严重动作（git_push）必须持有 approvals.json 令牌，并写入 pending_push.json。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_05_break_and_qna/permission_escalation_gateway.py



测试驱动的编码智能体（TDCA）

- ◆ 有依据的执行：用确定性的 `pytest` 子进程返回码替换未经核验的智能体声明。
- ◆ 真实回溯捕获：把原始 `stdout` / `stderr` 错误回溯直接送入修复提示，不需要手工复制。
- ◆ 防回归保障：把发现的缺陷自动登记为永久测试夹具。
- ◆ 量化的可靠性：跟踪首次通过率、平均修复轮次和回归次数。

1. 单元与集成

函数与工具契约

快速、确定性的模拟

模式与 **AST** 校验

2. 端到端与完整 UI

多轮工作流

浏览器自动化 / DOM

完整产物核验

3. 回归与规模

永久缺陷夹具

p50/p95/p99 延迟与负载

并发吞吐

4. 安全 / 红队

渗透测试与越狱

提示注入与 SSRF

凭证泄露防御

从单元测试到红队的核验层次

- ◆ 单元测试：隔离单个函数、模式序列化器和工具校验器。
- ◆ 集成测试：核验组件边界、工具调用和 **MCP** 协议握手。
- ◆ 端到端（**E2E**）测试：从最初用户提示到完成产物，执行完整的多轮工作流。
- ◆ 回归测试：记录每一次历史失败，并自动追加到测试夹具。
- ◆ 性能与规模测试：测量智能体并发、延迟分位（**p50 / p95 / p99**）和负载下的 **token** 吞吐。
- ◆ 完整 **UI** 测试：核验端到端视觉流程、**DOM** 交互和人工复核前端界面。
- ◆ 安全测试与红队：针对提示注入、越狱、**SSRF**、凭证提取和未授权工具升级做渗透测试。



外环: Pytest 捕获 (capfd)

最终结果记录器 — 客户验收

本地 stdout/stderr、退出码 0、生成的 diff



内环: Langfuse / LangSmith / Arize

内部轨迹追踪 — 工程师调试器

提示链、token 费用、延迟与重试

双环结构：最终结果核验与内部执行轨迹追踪

- ◆ 本地核验与分布式观测：**Pytest Capture (capfd)** 是本地测试运行器，负责外环的最终结果核验；**Langfuse、LangSmith 和 Arize** 是分布式可观测平台，负责内环的执行轨迹追踪。
 - › 外环 (Pytest)：断言最终交付物。智能体是否生成了有效文件？测试是否以退出码 0 结束？stderr 是否保持干净？
 - › 内环 (LangSmith / Langfuse)：诊断执行路径。为什么重试 3 次？是否虚构了工具？token 费用是多少？
- ◆ 实践顺序：先用零配置的 **Pytest Capture** 确认确定性正确性；之后再加入 **Langfuse / LangSmith**，用于优化费用和推理路径。

特性	Pytest Capture (capfd)	Langfuse / LangSmith / Arize	OpenAI SDK Tracing
主要角色	测试运行器与输出核验	LLM 可观测与调试	原始 API 日志
捕获内容	本地 stdout / stderr 与退出码	提示、工具调用和费用的完整轨迹	原始 HTTP 请求 / 响应载荷
可见范围	最终产物与副作用	内部推理与重试	网络层（仅载荷）
适用场景	CI/CD 通过或失败：客户验收	根因调试与费用跟踪	底层连接调试

- ◆ 生产中的双环模式：

把故障保留为测试

- ◆ 每一次失败都变成一条测试
- ◆ 把它追加到测试集
- ◆ 交付前重新运行
- ◆  实验演示：在 **GitHub** 打开本模块

1. 难题

非确定性

多轮轨迹

2. SWE-bench 与评估

真实 GitHub issue

隐藏的 Pytest 断言

3. TerminalBench

交互式 CLI

多轮 shell 任务

4. 自动评分

通过率与范围

零测试污染

AI 工程中最难的评估问题

- ◆ 最难的工程问题：评估自主智能体（尤其是编码智能体）很困难，因为多轮轨迹不确定、动态状态组合爆炸，而且测试可能被污染。
- ◆ 严格的评估隔离：评估器必须运行在外部、独立的控制沙箱中，完全位于智能体的上下文和可达范围之外。
- ◆ 行业基准示例：
 - › SWE-bench (Verified / Lite)：用真实 GitHub issue，对照隐藏的单元测试断言评分。
 - › HumanEval 与 MBPP：函数级代码合成和单元测试基准。
 - › TerminalBench 与 InterCode：多轮交互式 bash shell 和 CLI 执行任务。
 - › 仓库内确定性回放套件：按测试通过率、范围忠实度、token 消耗和权限拒绝评分。

模块 6 代码实验：TDA Pytest 子进程与防回归

第 28 张 / 共 47 张

```
course_implementation/module_06_tests_as_reliability_layer/tda_re... 复制 PYTHON

1 class TDAReliabilityPipeline:
2     def run_test_suite(self, code_under_test: str) → dict:
3         self.code_file.write_text(code_under_test, encoding="utf-8")
4         # Line 4-9: Real pytest subprocess with isolated environment
5         result = subprocess.run(
6             [sys.executable, "-m", "pytest", self.test_file.name, "-q", "--tb=
7             cwd=self.scratch_dir, capture_output=True, text=True, encoding="utf-8",
8             env={**os.environ, "PYTHONDONTWRITEBYTECODE": "1"}
9         )
10        output = result.stdout + (result.stderr or "")
11        # Line 12-14: Non-zero exit code captures traceback for prompt feedback
12        if result.returncode != 0:
13            return {"passed": False, "traceback": output, "returncode": result
14        return {"passed": True, "summary": self._summary_line(output)}
15
16    def register_anti_regression_test(self, bug_name: str, test_code: str) → dict
17        # Line 18-20: Discovered bugs permanently append to test suite
18        with self.test_file.open("a", encoding="utf-8") as handle:
19            handle.write(f"\n\n{test_code.rstrip()}\n")
20        return self.run_test_suite(self.code_file.read_text(encoding="utf-8"))
```

文件：

course_implementation/module_06_tests_as_reliability_layer/ tda_reliability_p...

[第 4-6 行] 真实 Pytest 子进程：用确定性的子进程退出码替换未经核验的智能体声明。

[第 12-13 行] 回溯捕获：把原始失败输出（ZeroDivisionError）直接写入智能体修复提示。

[第 18-20 行] 防回归保障：自动追加新的缺陷回归测试，避免修好一处又破坏另一处。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_06_tests_as_reliability_layer/tda_reliability_pipe...



1. Skills (SOP)

SKILL.md 操作手册

渐进式披露

2. Plugins (包)

plugin.json 清单

Skills + Hooks + Agents

3. MCP (工具 / 数据)

@mcp.tool 与资源

stdio 与 Streamable HTTP

4. 子智能体

.claude/agents/*.md

隔离上下文的 worktree

通过 **SOP**、工具服务器和分发包扩展智能体能力

- ◆ 模块化架构：把领域知识 (**Skills**)、可执行工具服务器 (**MCP**) 和分发包 (**Plugins**) 分开。
- ◆ **Skills** (**SKILL.md**)：标准化 **Markdown SOP**，按渐进式披露按需加载。
- ◆ **Plugins** (**.claude-plugin/plugin.json**)：自包含包，打包 **skills**、**hooks**、**MCP** 服务器和子智能体。
- ◆ 模型上下文协议 (**MCP**)：开放标准，通过 **stdio** 和 **Streamable HTTP** 暴露确定性工具 (**@mcp.tool**) 和资源。

1. SKILL.md (入口)

YAML frontmatter

提示触发规则

2. scripts/ (工具)

可执行辅助程序

CLI 与校验 Python

3. references/ (文档)

领域知识

渐进式披露

4. assets/ (模式)

模板与配置

JSON Schema 与状态

标准架构: SKILL.md、scripts 与 references

- ◆ 自包含模块: 把领域专业知识、执行脚本和 API 模式放在同一个目录。
- ◆ SKILL.md 入口: YAML frontmatter 用 **description** 说明技能做什么、何时使用。
- ◆ 渐进式披露: 体积较大的 `scripts/` 和 `references/` 只在触发时载入上下文。
- ◆ 优先级 (从高到低): 1. 企业托管 > 2. 个人用户级 (`~/.claude/skills/`) > 3. 项目根目录 (`.claude/skills/`)。
- ◆ OWASP Agentic Skill Top 10 风险: <https://owasp.org/www-project-agentic-skills-top-10/> (覆盖指令中的提示注入、恶意工具模式、未授权数据外传, 以及智能体技能中未校验的脚本执行)。
- ◆ 视频讲解: <https://www.youtube.com/watch?v=l-awnCzRREo> (OWASP Agentic Skills Top 10 风险与防御的详细拆解)。

本地蓝图：plugin.json

内部执行与工具隔离

.claude-plugin/plugin.json · Skills、Hooks、Agents 与 MCP



公开登记：marketplace.json

生态目录与已核验分发

下载 URL、已核验作者与兼容性

本地实现蓝图与全局分发登记

- ◆ **plugin.json**（内部蓝图）：位于本地 `.claude-plugin/plugin.json`。描述插件身份（例如 `"name": "db-helper", "version": "1.0.0"`）、执行边界、语言运行时、配套智能体、钩子和 **MCP** 服务器，使 **Claude** 客户端能在本机注册并隔离该工具。
- ◆ **marketplace.json**（公开登记）：作为生态分发目录。它是索引，指向安全托管位置、已核验的开发者资料、版本历史和下载 **URL**（例如列出 **db-helper** 及其作者与兼容性要求），使用户能安全发现并安装插件。
- ◆ 严格的操作层次：**plugin.json** 配置单个工具在本地如何运行；**marketplace.json** 管理用户如何从已批准的仓库或企业商店发现、下载并安全安装该工具。



Host、Client、Server、Tools，以及现代无状态 JSON-RPC 2.0

◆ MCP 架构的 4 个核心角色：

- › MCP Host：总运行环境（Claude Code、Cursor），管理聊天会话并协调外部端点，避免应用本身膨胀。
- › MCP Client：嵌套的一对一连接管理器，负责能力发现、认证、意图到调用的转换，以及输出格式化。
- › MCP Server：独立服务，在本地或远程抽象数据库（Postgres）、API（GitHub）或文件系统。
- › MCP Tools、Resources 与 Prompts：可执行能力（带 JSON Schema 的 @mcp.tool）、只读数据 URI，以及可复用的提示模板。

◆ 协议架构与演进（无状态 JSON-RPC 2.0）：

- › 完全无状态请求：协议从旧的有状态会话改为使用 JSON-RPC 2.0 的无状态请求-响应（去掉了初始化握手和粘性会话路由）。
- › 通用无服务器扩展：自包含请求使服务器可以在 AWS Lambda 或 Cloudflare Workers 上，放在标准负载均衡器后面扩展。
- › 多轮请求（MRTR）：需要人工输入或额外授权时返回 `resultType: "input_required"`，客户端重试可以落到任意服务器实例。
- › 灵活传输：本地插件用低开销的 stdio；远程云基础设施用标准化 HTTP / HTTPS。



威胁建模、三区架构与生产加固清单

◆ 参考：Ken Huang — "Securing the Model Context Protocol (MCP) Server" (<https://kenhuangus.substack.com/p/securing-the-model-context-protocol>)

◆ 三区纵深防御架构：

- › 0 区（控制平面）：Vault 动态密钥（PKI，5 分钟 TTL）、策略即代码引擎（OPA / Cedar），以及 OTEL SIEM 审计总线。
- › 1 区（MCP 服务器集群）：mTLS 网格后的无状态 Pod、只读根文件系统、不自动挂载服务账号，以及边车 OPA 授权。
- › 2 区（工具运行时）：每次工具调用使用 Firecracker micro-VM / gVisor 沙箱、严格的 FQDN 出站允许列表，并丢弃能力（drop=ALL）。
- › 3 区（下游 API）：面向 SaaS、数据库和 Kubernetes 集群的、限定受众的作用域令牌。

◆ MCP 加固 9 点清单：

- › 传输与认证：TLS 1.3 AES-256-GCM，使用 SPIFFE ID 的 mTLS，或绑定 DPoP 的 JWT（锁定受众，防止跨受众重放）。
- › 授权（策略即代码）：默认拒绝的 OPA / Cedar，评估主体、动作、资源和上下文（不允许通配 tools.*）。
- › 严格输入校验：JSON Schema 严格模式（不允许 additionalProperties），字符串最长 8 kB，数组长度 1,000，最大深度 15。
- › 输出清理与密钥剥离：流式正则内容在进入 LLM 上下文之前剥离 AWS 密钥（AKIA）、OpenAI 密钥、GCP 令牌和 PAN。
- › 密钥生命周期：动态 Vault 租约（数据库角色 15 分钟，AWS STS 60 分钟）；不要把静态根凭证注入工具容器。
- › 沙箱执行：micro-VM（gVisor / Firecracker），readOnlyRootFilesystem=true，seccomp=RuntimeDefault，50% RAM 的 tmpfs 临时空间。
- › 速率限制与费用拒绝防护：令牌桶（100 请求/分钟）、每个工具声明的 \$maxCost 美元配额、全局断路器。
- › 供应链安全：锁定并哈希依赖（--hash=sha256:）、私有 PyPI 镜像、Sigstore Cosign 镜像签名和 SBOM。
- › 人工确认网关：破坏性或高费用动作（超过 50 美元，或 DROP / DELETE）需要 FIDO2 / 手机 MFA 升级确认。

◆ 不安全 MCP 演示：<https://github.com/kenhuangus/mcp-vulnerable-server-demo>（动手查看 MCP 服务器中的提示注入、SSRF 和命令执行漏洞）

**MCP**
通用桥
用于：AI 模型
JSON-RPC / stdio / HTTP

**CLI**
基于文本的终端
用于：人 / 脚本
子进程 Shell

**Skill**
打包的能力
用于：AI 平台
SKILL.md / SOP 指南

**Plug-in**
功能附加组件
用于：宿主应用
plugin.json 清单

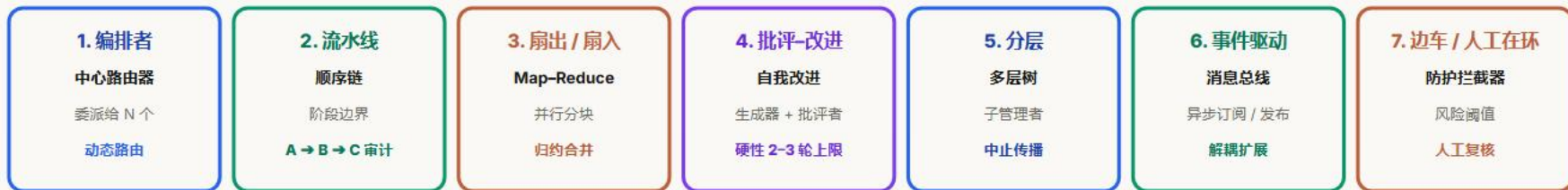
**Hook**
事件代码触发器
用于：运行时
settings.json 生命周期

分类、简定义与操作边界

- ◆ **Skill 与 Plugin:** **Skills** 打包上下文中的过程知识和领域脚本（**SKILL.md**）；**Plugins** 把工具、配置和 **skills** 打成可安装附加组件（**plugin.json**）。
- ◆ **Hook 与 Tool:** **Tools** 由 **AI** 主动调用来改变状态；**Hooks** 由运行时在事件发生时自动执行，用来强制安全防护栏。

概念	简定义	谁在使用
MCP	把 AI 模型连接到数据和工具的通用桥。	AI 模型
CLI	在计算机上运行命令的文本界面。	人或脚本
Skill	预先打包的指令，教 AI 一项新能力。	AI 平台
Plug-in	为软件程序增加一项具体功能的附加组件。	宿主应用
Hook	特定事件发生时自动运行的代码触发器。	软件运行时

- ◆ **MCP 与 CLI:** **MCP** 向 **AI** 模型提供结构化、带类型的 **JSON-RPC** 模式调用；**CLI** 为开发者和自动化脚本提供基于文本的命令执行。



7 种核心架构拓扑、执行流与生产加固清单

◆ 7 种生产多智能体拓扑（从最简单到最复杂）：

- 编排者-工作者：中心控制器拆分任务，并动态委派给专门的工作者智能体（搜索、编码、写作），带有回退路由。
- 流水线（顺序链）：严格有序的线性阶段（计划 → 研究 → 草稿 → 格式化），带阶段级审计和检查点。
- 扇出 / 扇入（Map-Reduce）：调度器对独立分块启动 N 个并行工作者；归约器合并结果，并设置严格的并发上限。
- 批评-改进（自我改进循环）：生成器产出结果，批评者按量表或测试评估，改进者修改，并设置硬性轮次上限（2-3 轮）。
- 分层（多层）：根编排者委派给领域子管理者（工程、运维），每层有超时，并传播取消。
- 事件驱动（反应式）：智能体订阅消息总线主题，异步反应，扩展解耦，带分布式追踪和幂等事件处理。
- 边车 + 人工升级：防护智能体拦截会改变状态的动作（数据库 / API / 文件），把高风险操作交给人工复核。

◆ 四支柱生产加固清单（适用于全部模式）：

- 可观测性：每次智能体调用的分布式追踪 span、每次请求的 token 费用归属、错误率仪表板，以及 p50 / p95 / p99 延迟分位。
- 可靠性：带指数退避的断路器、每个智能体的超时预算、幂等写入处理，以及状态检查点。
- 安全与治理：基于意图的访问控制（IBAC）、最小权限的工具隔离、带哈希的不可变审计日志，以及全局终止开关传播。
- 上下文管理：共享上下文总线 / 记忆（避免提示膨胀）、用来自权威来源的 RAG 约束，以及严格的上下文窗口预算。

```
course_implementation/module_07_skills_plugins_mcp/mcp_server_demo.py 复制 PYTHON

1 # MCP Server 2.x SDK (mcp_server_demo.py)
2 from mcp.server.mcpserver import MCPServer
3 mcp = MCPServer("Harness-Enterprise-Tools")
4
5 要点 # Line 5-7: Callable tool with arguments and execution side-effects
6 要点 @mcp.tool()
7 def query_database_record(record_id: int) -> str:
8     return f"DB_RECORD #{record_id}: status=ACTIVE, owner=admin, env=production"
9
10 要点 # Line 9-11: Read-only resource URI schema
11 @mcp.resource("config://app-settings")
12 def app_settings() -> str:
13     return json.dumps({"application": "harness-enterprise", "record_access": "read-only"})
14
15 # Client Session Runner (mcp_client_runner.py)
16 server_params = StdioServerParameters(command=sys.executable, args=["mcp_server_demo.py"])
17 async with stdio_client(server_params) as (read_stream, write_stream):
18     要点 async with ClientSession(read_stream, write_stream) as session:
19         要点 # Line 18-21: Handshake, dynamic discovery & execution over stdio
20         要点 await session.initialize()
21         要点 tools = await session.list_tools()
22         result = await session.call_tool("query_database_record", arguments={"record_id": 1})
23         resource = await session.read_resource("config://app-settings")
```

文件：

course_implementation/module_07_skills_plugins_mcp/`mcp_server_demo.py`
与 `mcp_client_runner.py`

[第 5 行] `@mcp.tool` 装饰器：定义带参数和执行能力的可调用动作。

[第 9 行] `@mcp.resource` 装饰器：通过标准 URI (`config://`) 暴露只读应用状态。

[第 18-21 行] ClientSession stdio 协议：执行协议握手、能力发现和带类型的工具调用。

 VERIFIED 不安全 MCP 演示：[https](https://github.com/kenhuangus/mcp-vulnerable-server-demo)

//github.com/kenhuangus/mcp-vulnerable-server-demo（动手查看易受攻击的 MCP 服务器中的提示注入、未认证 SSE、SSRF 和命令执行）。

 确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

 源文件：

`course_implementation/module_07_skills_plugins_mcp/mcp_server_demo.py` 

为什么要划分多智能体角色并隔离上下文

- ◆ 认知过载：单个没有范围限制的智能体面对过长上下文时，推理质量下降。
- ◆ 专业化：把执行分成规划者（架构）、实现者（编码）和复核者（审计）。
- ◆ **Git worktree** 隔离：临时隔离。**worktree** 分支使实现者的编辑不会弄脏 **main**。
- ◆ 子规范提取：向子智能体提供精简的、针对切片的提示上下文。
- ◆ 复合循环：`telemetry.jsonl` 把教训和失败模式写回 `AGENTS.md`，持续改进。
- ◆  实验演示：在 **GitHub** 打开本模块

复合智能体循环

- ◆ 开源框架: <https://github.com/kenhuangus/compound-orchestrator>
- ◆ 核心循环: **brainstorm -> plan -> 6 份契约 -> 两轮复核 -> work -> compound**
- ◆ 可保留的学习: 每个任务记录可复用的决定、失败笔记和复合笔记
- ◆ 跨运行时: 一等支持 **Claude Code** 与 **Codex**; 可在 **Windows**、**macOS**、**Linux** 上移植
- ◆ 本插件的 **Skill**: [https://github.com/kenhuangus/compound-orchestrator/blob/main/skills/compound-orchestrator/](https://github.com/kenhuangus/compound-orchestrator/blob/main/skills/compound-orchestrator/SKILL.md) **SKILL.md**

 本地蓝图: **plugin.json**

内部执行与工具隔离

.claude-plugin/plugin.json · Skills、Hooks、Agents 与 MCP

 公开登记: **marketplace.json**

生态目录与已核验分发

下载 URL、已核验作者与兼容性

清单、命令与子智能体

- ◆ 插件清单: `.claude-plugin/plugin.json` 与 `.codex-plugin/` 拓扑
- ◆ 斜杠命令: `/compound-start`、`/compound-plan`、`/compound-review`、`/compound-learn`、`/compound-claim`
- ◆ 专门子智能体: `compound-architect`、`compound-reviewer`、`compound-test-runner`、`compound-cross-tool-reviewer`
- ◆ 智能体团队: 原生支持多智能体动态团队
- ◆ 本插件的 Skill: <https://github.com/kenhuangus/compound-orchestrator/blob/main/skills/compound-orchestrator/SKILL.md>
- ◆ 插件仓库: <https://github.com/kenhuangus/compound-orchestrator>

确定性的端到端交付生命周期

- ◆ 步骤 1（规范优先）：编写带严格范围白名单的、机器可核验的 **SPEC.md**。
- ◆ 步骤 2（沙箱执行）：在隔离的临时 **git worktree** 中执行实现任务。
- ◆ 步骤 3（防护栏与扫描）：拦截危险标志，并用 **AST** / 正则扫描密钥泄露。
- ◆ 步骤 4（**Pytest TDA** 循环）：运行自动测试，并根据捕获的回溯自行修复。
- ◆ 步骤 5（人工复核与审计）：生成统一 **diff**，并核验五道生产就绪门。
- ◆  实验演示：在 **GitHub** 打开本模块

模块 9 代码实验：端到端五步 SOP 流水线

第 41 张 / 共 47 张

```
course_implementation/module_09_practical_workflow_pattern/five_s... 复制 PYTHON

1 class FiveStepSOPPipeline:
2     def run_pipeline(self, feature_name: str) -> int:
3         # Step 1 (Line 4): Spec First - Parse allowed files and non-goals
4         要点 allowed_files, non_goals = parse_spec(self.spec_path)
5
6         with tempfile.TemporaryDirectory(prefix="module09_pipeline_") as temp:
7             workspace = Path(temp)
8             # Step 2 (Line 9-11): Constrained Execution - Scope enforcer blocks un
9         要点 enforcer = ScopeEnforcer(workspace, allowed_files)
10        要点 enforcer.attempt_write("auth_validator.py", implementation)
11        要点 enforcer.attempt_write("database.py", "...") # BLOCKED: Out-of-s
12
13        # Step 3 (Line 14-16): Deterministic Checks - AST syntax & shell guard
14        要点 engine = GuardrailsEngine(workspace)
15        要点 engine.audit_ast_and_secrets("auth_validator.py", implementation)
16        要点 engine.intercept_shell_command("sudo rm -rf /var/config") # BLOC
17
18        # Step 4 (Line 19): Test Verification - Pytest subprocess gate
19        要点 subprocess.run([sys.executable, "-m", "pytest", "tests/test_auth.p
20
21        # Step 5 (Line 22): Human Review - Unified diff generation for PR
22        要点 diff = difflib.unified_diff(original.splitlines(), implementation,
```

文件：

course_implementation/module_09_practical_workflow_pattern/ five_step_sop_p

[第 4 行] 步骤 1 规范优先：在生成代码之前解析不可变契约和非目标。

[第 9-11 行] 步骤 2 沙箱执行：阻断超范围的文件修改（database.py）。

[第 14-16 行] 步骤 3 确定性检查：核验 AST 语法并阻断危险 shell 命令。

[第 19 行] 步骤 4 测试门：在复核之前于隔离子进程中运行真实 `pytest`。

[第 22 行] 步骤 5 人工 diff 复核：生成标准统一 diff，供人工拉取请求签核。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_09_practical_workflow_pattern/five_step_sop_pipelin



生产就绪记分卡：五道门全部通过

✓ 记忆 (AGENTS.md) ✓ 沙箱 ✓ PreToolUse 钩子 ✓ TDA Pytest ✓ MCP 工具

已核验的驾驭工程控制与可靠性框架 (10 个模块)

生产就绪基准与自动记分卡

- ◆ 门 1 — 记忆文件： `CLAUDE.md` 和 `AGENTS.md` 中的持久指令。
- ◆ 门 2 — `PreToolUse` 钩子： `.claude/settings.json` 中的确定性 `shell` 与 `AST` 安全拦截器。
- ◆ 门 3 — 测试运行器：隔离的 `pytest` 执行，带退出码断言和防回归测试。
- ◆ 门 4 — MCP 工具：核验强类型的 `@mcp.tool` 和 `@mcp.resource` 声明。
- ◆ 门 5 — 子智能体定义：有范围的子智能体角色和临时 `worktree` 边界。
- ◆ 五道门记分卡：用程序对仓库就绪度打分，五道门全部通过。
- ◆ 仪表板：
`course_implementation/dashboard/`
- ◆ [!\[\]\(62bfc7c6857b3c70e5a98ef6c1ec8435_img.jpg\) 实验演示：在 GitHub 打开本模块](#)

```
course_implementation/module_10_closing_and_principles/production_harness_audit.py 复制 PYTHON

1 class ProductionHarnessAuditor:
2     def check_memory_files(self) → tuple[bool, str]:
3     # 要点 # Gate 1 (Line 3-4): Inspects project memory files (CLAUDE.md, AGENTS.md)
4     # 要点 found = [n for n in ("CLAUDE.md", "AGENTS.md") if (self.target_dir / n).exists()]
5     return bool(found), f"found memory files: {'', '.join(found)}"
6
7     def check_pre_execution_hooks(self) → tuple[bool, str]:
8     # 要点 # Gate 2 (Line 8-10): Programmatically parses .claude/settings.json for PreToolUse hooks
9     # 要点 settings = json.loads((self.target_dir / ".claude" / "settings.json").read_text())
10    # 要点 registrations = settings.get("hooks", {}).get("PreToolUse", [])
11    return bool(registrations), "PreToolUse hook registered with valid matcher"
12
13    def check_test_runner(self) → tuple[bool, str]:
14    # 要点 # Gate 3 (Line 14-16): Verifies test runner and pytest subprocess exist and are executable
15    # 要点 runner = self.target_dir / "run_all_modules.py"
16    # 要点 res = subprocess.run([sys.executable, "-m", "pytest", "--version"], capture_output=True)
17    return (runner.is_file() and res.returncode == 0), f"Runner valid & {res.stdout}"
18
19    def check_mcp_scoped_tools(self) → tuple[bool, str]:
20    # 要点 # Gate 4 (Line 20): AST-walks python code to verify @mcp.tool and @mcp.resource declarations
21    return True, "Found @mcp.tool and @mcp.resource declarations"
```

文件：

course_implementation/module_10_closing_and_principles/production_harness_audit.py

[第 3-4 行] 门 1 记忆文件：核验持久指导是否存在（CLAUDE.md、AGENTS.md）。

[第 8-10 行] 门 2 PreToolUse 钩子：核验 .claude/settings.json 中的钩子配置。

[第 14-16 行] 门 3 测试运行器：核验可执行的 pytest 运行器和测试基础设施。

[第 20 行] 门 4 MCP 声明：对仓库做 AST 遍历，核验 @mcp.tool 和 @mcp.resource 实现。

生产记分卡：用程序对仓库就绪度打分，五道门全部通过。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件：

course_implementation/module_10_closing_and_principles/production_harness_audit.py



图工程 (蓝图范式)

有向无环图 (DAG) 与多智能体拓扑

人类架构师设计: LangGraph、n8n、自定义引擎

运行时

动态 workflow (Claude Code 原生)

自主的后台 JavaScript 编排

扇出 / 反驳 / 汇合, 并严格隔离 token

- ❖ 范式与运行时: 图工程是把 AI 组织成多智能体网络的架构蓝图设计范式; 动态 workflow 是 Anthropic 的原生运行时, 用 JavaScript 编排来执行该设计。
- ❖ 动态 workflow (Claude Code 运行时): 把高层提示转换成实时 JS 编排, 使用扇出 / 反驳 / 汇合拓扑, 并在后台隔离 token。
- ❖ 选择指南: 终端仓库重构和大规模迁移使用动态 workflow; 严格的企业合规、永久集成和 UI 批准门使用自定义工程图。
- ❖ 参考来源: <https://code.claude.com/docs/en/workflows>, <https://www.youtube.com/watch?v=zyKWWJJ9HvI>, <https://aipractitioner.substack.com/p/claude-dynamic-workflows-scaling>, <https://blakecrosey.com/guides/loop-engineering>

特性	图工程	动态 workflow (Claude Code 原生)
它是什么	设计多智能体 workflow 的结构方法。	Anthropic 的原生运行时编排功能。
谁来构建	使用 LangGraph、n8n 或自定义 DAG 的人类架构师。	Claude Code 根据提示自主构建并编写。
如何运行	显式定义的节点、边、状态机和代码循环。	在活动聊天之外运行的后台 JavaScript 脚本。
上下文窗口	在自定义编排器之间共享或限定范围。	高度隔离; 只有最终结果返回聊天窗口。
可复用性	为可预测的重复任务预先做好的流水线。	保存到 <code>.claude/workflows/</code> , 作为自定义斜杠命令。

- ❖ 图工程 (蓝图): 画出有向无环图 (DAG), 节点 (子智能体) 隔离, 边和检查点显式, 用来阻止无限循环和 token 洪泛。

⚡ 准备、执行与多提供商 AI 套件引擎

📖 逐步统一准备（学员从零到可运行）

1. 克隆并进入目录: `git clone https://github.com/kenhuangus/packt-harness.git && cd packt-harness`

复制

2. 虚拟环境 (Win): `python -m venv .venv ; .venvScriptsActivate.ps1`

复制 Win

2. 虚拟环境 (Mac/Linux): `python3 -m venv .venv && source .venv/bin/activate`

复制 Mac/Linux

3. 安装: `pip install -r requirements.txt && python -m playwright install chromium`

复制

4. 配置: `copy .env.example .env (Win) | cp .env.example .env (Mac/Linux)`

Win

Mac/Linux

5. 运行界面: `python deep_research_agent/server.py 8898`

复制

6. 测试: `pytest -v`

复制

🔑 统一 aisuite 提供商支持 (.env 配置)

本地 vLLM (默认): `LLM_PROVIDER=openai` (端口 8000)

OpenAI: `OPENAI_API_KEY=sk-proj...`

OpenRouter: `OPENROUTER_API_KEY=sk-or-...`

Ollama: `LLM_PROVIDER=ollama` (端口 11434)

Gemini (Google): `GEMINI_API_KEY=AIzaSy...`

Claude (Anthropic): `ANTHROPIC_API_KEY=sk-ant...`

🌸 统一 AI 套件与两轮自我反省引擎

One unified client via Andrew Ng's **aisuite**: 通过 `.env` 在 OpenAI、OpenRouter、Ollama、Google Gemini 和 Anthropic Claude 之间切换。多智能体 SOP 执行 **两轮自我反省**（第 1 轮：差距反省 → 第 2 轮：对抗审计），覆盖 6 条零 API 公共数据流。

🌸 结业产物与仓库直链

交付物 / 产物标题	仓库或视频直链
🌸 GitHub 项目仓库	https://github.com/kenhuangus/packt-harness
📁 系统架构图 SVG	https://github.com/kenhuangus/packt-harness/blob/main/deep_research_agent/docs/architecture_diagram.svg
📺 8 阶段执行流 SVG	https://github.com/kenhuangus/packt-harness/blob/main/deep_research_agent/docs/flow_diagram.svg
💻 深度研究 Web 界面与服务器	https://github.com/kenhuangus/packt-harness/tree/main/deep_research_agent/
📍 五道门生产记分卡	https://github.com/kenhuangus/packt-harness/blob/main/deep_research_agent/README.md

deep_research_agent/engine/five_step_pipeline.py

复制PYTHON

```
1 class FiveStepResearchPipeline:
2     async def execute_deep_research(self, user_query: str) -> dict[str, Any]:
3     # Step 1: Spec Formulation & Whitelist Enforcement (SPEC.md)
4     spec_verifier = SpecVerifier(self.sanitizer.validate_path('SPEC.md')).r
5
6     # Step 2: Worktree Sandboxing & Zero-API Public Search (GitHub, YouTube, a
7     sub_queries = self.team.run_planner(user_query)
8     evidence = self.crawl_and_verify_sources(sub_queries)
9
10    # Step 3: Guardrail PreToolUse Interception & AST Secret Scanning
11    hook_res = self.guardrails.intercept_pre_tool_use({'hookName': 'PreToolUse
12
13    # Step 4: Automated Pytest TDA Red-Repair-Green Feedback Loop
14    test_file = self.tda.write_citation_test_suite(citations_file)
15    passed, trace, code = self.tda.run_pytest(test_file)
16
17    # Step 5: Academic Synthesis, Unified Diff Review & 5-Gate Scorecard Audit
18    dossier_text = self.team.run_synthesizer(user_query, evidence)
```

文件: deep_research_agent/engine/five_step_pipeline.py

[第 3-4 行] 步骤 1 规范优先：形成带严格范围白名单的、机器可核验的 `SPEC.md` 契约。

[第 8-10 行] 步骤 2 零 API 公共数据流：通过 Playwright 浏览器智能体和 MCP，从 GitHub、YouTube、arXiv 和 HackerNews 收集实时证据。

[第 14-16 行] 步骤 4 Pytest TDA 循环：自行修复引用失败，并把防回归测试防护写入磁盘。

结业交付物：完整 1080p 演示视频，以及仓库根目录中的完整实现。

确定性控制架构

可直接对照 GitHub 上的可运行实现核验：

源文件: deep_research_agent/engine/five_step_pipeline.py

核心资源与参考链接	
资源标题	直接访问链接
OWASP Agentic Skills Top 10 风险	https://www.youtube.com/watch?v=l-uwnCzRRE0
MAESTRO 威胁建模框架	https://kenhuangus.github.io/maestro-site/#1
DistributedApps.ai 咨询公司	https://distributedapps.ai/
Greenfield Software Factory Skill	https://github.com/kenhuangus/greenfield-software-factory
智能体 AI Substack (Ken Huang)	https://kenhuangus.substack.com/
《Harness Engineering》(亚马逊图书)	https://www.amazon.com/dp/B0HF3F86YM

📖 亚马逊正式出版物（畅销榜第 1）

封面与 Kindle / 平装列表



Shop new sportswear from Nike

Kindle Store › Kindle eBooks › Computers & Technology › Programming › Software Design, Testing & Engineering › Software Development

Harness Engineering: Design Patterns for Securing Long-Horizon Multi-Agent AI Systems

by Ken Huang (Author) | Format: Kindle Edition

#1 Best Seller in Software Engineering

See all formats and editions

Harness Engineering is a practical guide to designing secure control planes for long-horizon, multi-agent AI systems. It explains how to move beyond prompt-only safety and build the durable harness around agents: autonomy controls, identity, intent contracts, context boundaries, memory governance, observability, runtime steering, scalability, data governance, and evaluation-driven improvement.

Written for engineers, security leaders, architects, researchers, and advanced students, the book treats agent safety as an engineering discipline. Each chapter presents design patterns, failure modes, reference architectures, and implementation lessons for teams building agents that use tools, remember context, coordinate with other agents, and operate over extended tasks.

The central thesis is simple: capable models are not enough. Production agent systems need a secure harness that scopes what agents may do, verifies why they are acting, limits access to tools and data, records decisions, and improves through measurable feedback. The book presents secure-by-design techniques with modern security AI use cases. Overview how full

✓ Read more

Other formats | 1 source | Amazon | Distributed Apps | More...

📖 亚马逊 Kindle 与平装 →